# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

## ASSESSMENT TOOL 2 – Dry Run Evaluation

| | | | |
|---|---|---|---|
| **Course Code:** | CSA17-16-16 | **Course Title:** | Artificial Intelligence |
| **CO Assessed:** | CO1 – Apply (BL3) | **Assessment:** | Assessment Tool 2 – Dry Run Evaluation |
| **Weightage:** | 35% of CIA | **Duration / Marks:** | 60 mins / 100 marks |
| **CO1 Statement:** | Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3) | | |
| **Student Name:** | M. SRILEKHA | **Reg. No.:** | 1919260007 |
| **Section / Batch:** | | **Date:** | 10 October 2026 |

**Code of Conduct**

*I, M. Srilekha (Reg. No. 1919260007), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: M. Srilekha

**Instructions**

- Read each question carefully; answer all four questions (25 marks each, 100 in total).
- Show every dry-run step: the initial queue, stack, priority queue or belief state, each iteration, your assumptions and a clear final answer, and finish with the interpretation asked for.
- Follow the conventions stated in each question (alphabetical expansion, explored set, goal test on removal from the frontier, tie-breaking) and present tables neatly.
- Marks follow the Dry Run Evaluation rubric: understanding of the search problem, state expansion and dry run execution, application of the search algorithm, accuracy of the solution path, and presentation and logical reasoning.
- Upload in GitHub and GCR (question paper with Reg. No., Name, answers, rubrics and evaluation page at the end). Duration: 1 hour (15 minutes per question).

| Section / Topic | Focus | Question Nos. |
|---|---|---|
| **1. Foundations and History of AI** | Four approaches to AI, Turing Test, eras and milestones | **Q2** |
| **2. Intelligent Agents: Agents and Environments** | Agent, percepts, actions, agent function and program, PEAS | **Q1** |
| **3. The Concept of Rationality** | Performance measure, percept sequence, rational vs omniscient | **Q1** |
| **4. The Nature of Environments** | Task-environment properties and their classification | **Q1** |
| **5. The Structure of Agents** | Simple reflex and model-based reflex agent programs | **Q1** |
| **6. Problem-Solving Agents** | Goal formulation, problem formulation, search, execution | **Q2** |
| **7. Problem Formulation** | States, initial state, actions, goal test, path cost; belief states | **Q2, Q4** |
| **8. Searching for Solutions** | Search graph, frontier, explored set, node expansion | **Q3** |

NAME: M. Srilekha

REG NO: 1919260007

Q1. Agents, Environments and Rationality

(a) PEAS Description

PEAS stands for Performance Measure, Environment, Actuators and Sensors.

| PEAS | Description |
|---|---|
| Performance Measure | +10 for each square cleaned and −1 for each movement. |
| Environment | Two squares, A and B, both initially dirty. |
| Actuators | Suck, Left, Right and NoOp. |
| Sensors | Detect the current location and the status of the current square. |

(b) Classification of Task Environment

| Property | Classification |
|---|---|
| Observability | Partially observable |
| Number of agents | Single-agent |
| Determinism | Deterministic |
| Episodic/Sequential | Sequential |
| Static/Dynamic | Static |
| Discrete/Continuous | Discrete |

(c) Dry Run of Both Agents for 6 Time Steps

Initial State: Location A, Square A = Dirty, Square B = Dirty.

Agent X – Simple Reflex Agent

| Step | Percept | Action | Result |
|---|---|---|---|
| 1 | [A, Dirty] | Suck | A becomes Clean |
| 2 | [A, Clean] | Right | Moves to B |
| 3 | [B, Dirty] | Suck | B becomes Clean |
| 4 | [B, Clean] | Left | Moves to A |
| 5 | [A, Clean] | Right | Moves to B |
| 6 | [B, Clean] | Left | Moves to A |

Agent Y – Model-Based Reflex Agent

| Step | Percept | Action | Result |
|---|---|---|---|
| 1 | [A, Dirty] | Suck | A becomes Clean |
| 2 | [A, Clean] | Right | Moves to B |
| 3 | [B, Dirty] | Suck | B becomes Clean |
| 4 | [B, Clean] | NoOp | Both squares are clean |
| 5 | [B, Clean] | NoOp | Remains at B |
| 6 | [B, Clean] | NoOp | Remains at B |

(d) Calculation of Total Performance

Agent X:
Cleaning score =\(2 \times 10 = 20\)
Movement penalty =\(4 \times (-1) = -4\)
Total Performance =\(20 - 4 = \boxed{16}\)
Agent Y:
Cleaning score =\(2 \times 10 = 20\)
Movement penalty =\(1 \times (-1) = -1\)
Total Performance =\(20 - 1 = \boxed{19}\)
(e) Interpretation

1. Which agent is rational?
Agent Y is more rational because it remembers the status of both squares and avoids unnecessary movements. It achieves a higher performance score of 19 compared with Agent X's score of 16.
2. Why do the agents act differently for the same percept [B, Clean]?
Agent X is a simple reflex agent that considers only the current percept. Therefore, it moves left.
Agent Y is a model-based reflex agent that remembers that both squares are clean. Therefore, it performs NoOp.
3. Difference between a rational agent and an omniscient agent
Rational Agent: Selects the best action based on available information and expected performance.

Omniscient Agent: Knows the actual outcomes of actions in advance.

Conclusion: Agent Y performs better because it uses an internal state to avoid unnecessary movements and achieves a higher performance score.

QUESTION 2: Foundations and History of AI; Problem-Solving Agent and Problem Formulation

Part A (a): Four Approaches to AI and History of AI (8 Marks)

Complete the table as follows:

System / Idea Approach to AI Era / Milestone

| | | |
|---|---|---|
| Turing Test proposed (1950) | Acting Humanly | 1950 – Turing Test |
| General Problem Solver (Newell and Simon, 1957) | Thinking Humanly | 1950s – Early AI and cognitive simulation |
| Aristotle's syllogisms and formal logic | Thinking Rationally | Ancient Greece – Foundations of Logic |
| An agent that does the right thing to maximise expected performance | Acting Rationally | Modern AI – Rational Agent Approach |

Explanation

1. Acting Humanly: The machine behaves like a human. The Turing Test checks whether a machine can imitate human conversation.

2. Thinking Humanly: The machine tries to think like a human and solve problems using human-like reasoning.

3. Thinking Rationally: The machine uses logical rules to reach correct conclusions.

4. Acting Rationally: The machine chooses the best action to achieve its goal and maximise expected performance.

---

Part B (b): Problem Formulation (6 Marks)

Problem: Water Jug Problem

There are two jugs:

Jug X capacity = 4 litres.

Jug Y capacity = 3 litres.

Initially, both jugs are empty.

Goal: Get exactly 2 litres of water in Jug X.

1. State Representation

Represent the state as (x, y), where:

x = amount of water in Jug X.

y = amount of water in Jug Y.

For example, (2, 1) means Jug X contains 2 litres and Jug Y contains 1 litre.

2. Initial State

Both jugs are empty.

Initial state = (0, 0)

3. Actions

The agent can perform these actions:

1. Fill Jug X completely.

2. Fill Jug Y completely.

3. Empty Jug X.

4. Empty Jug Y.

5. Pour water from X into Y.

6. Pour water from Y into X.

4. Transition Model

The transition model describes the result of each action.

Fill X: (x, y) → (4, y)

Fill Y: (x, y) → (x, 3)

Empty X: (x, y) → (0, y)

Empty Y: (x, y) → (x, 0)

Pour X into Y until X is empty or Y is full.

Pour Y into X until Y is empty or X is full.

5. Goal Test

The goal is reached when Jug X contains exactly 2 litres.

Goal test: x = 2

6. Path Cost

Assume each action costs 1.

Path cost = Total number of actions.

---

Part C (c): Successor States (3 Marks)

Find all successor states of (0, 0) and (4, 0). Do not include actions that leave the state unchanged.

1. Successor states of (0, 0)

Action Resulting State

Fill Jug X (4, 0)
Fill Jug Y (0, 3)

Emptying either jug or pouring water does not change the state, so those actions are excluded.

Answer: (4, 0), (0, 3)

2. Successor states of (4, 0)

Action Resulting State

Fill Jug Y (4, 3)
Empty Jug X (0, 0)
Pour X into Y (1, 3)

When Jug X has 4 litres and Jug Y is empty, pouring 3 litres into Jug Y leaves 1 litre in Jug X.

Answer: (4, 3), (0, 0), (1, 3)

---

Part D (d): State Space and Shortest Solution (3 Marks)

1. Number of States

Jug X can contain 0, 1, 2, 3, or 4 litres.

Jug Y can contain 0, 1, 2, or 3 litres.

Therefore,

$$
\text{Number of states} = (4+1)(3+1)
$$

$$
= 5 \times 4 = \boxed{20 \text{ states}}
$$

\]

2. Shortest Solution

The shortest solution is:

Step Action State (X, Y)

0 Initial state (0, 0)
1 Fill Jug Y (0, 3)
2 Pour Y into X (3, 0)
3 Fill Jug Y (3, 3)
4 Pour Y into X until X is full (4, 2)
5 Empty Jug X (0, 2)
6 Pour Y into X (2, 0)

Goal reached: Jug X contains exactly 2 litres.

Path cost = 6 actions.

---

Part E (e): Four Phases of a Problem-Solving Agent (5 Marks)

1. Goal Formulation

The agent decides what it wants to achieve.

Goal: Get exactly 2 litres of water in Jug X.

2. Problem Formulation

The agent defines the initial state, possible actions, transition model, goal test, and path cost.

Initial state: (0, 0)
Goal state: Any state where X = 2.

3. Search

The agent searches for a sequence of actions that reaches the goal.

For example:

(0, 0) → (0, 3) → (3, 0) → (3, 3) → (4, 2) → (0, 2) → (2, 0)

4. Execution

The agent performs the actions in the solution sequence one by one.

It can execute the solution without looking at new percepts because the problem is deterministic, the actions have predictable results, and the agent assumes the jugs behave exactly as described.

Conclusion

A problem-solving agent formulates a goal, defines the problem, searches for a solution, and executes the selected actions to reach the goal

QUESTION 3: Searching for Solutions – Uninformed Search Strategies

Given:

Start node = S

Goal node = G

Use BFS, DFS and Uniform-Cost Search (UCS).

Expand neighbours alphabetically.

Use an explored set for graph search.

Test the goal when a node is removed from the frontier.

Given Graph

| Edge | Cost | Edge | Cost |
|---|---|---|---|
| S–A | 1 | B–D | 1 |
| S–B4 | | C–G | 6 |
| A–C | 2 | D–G | 2 |
| A–D | 5 | | |

---

Part A (a): Breadth-First Search (BFS) — 6 Marks

BFS uses a queue (FIFO): First In, First Out.

Initial queue: [S]

| Iteration | Node Dequeued | Queue after Expansion | Explored Set |
|---|---|---|---|
| 1 | S | [A, B] | {S} |
| 2 | A | [B, C, D] | {S, A} |
| 3 | B | [C, D] | {S, A, B} |
| 4 | C | [D, G] | {S, A, B, C} |
| 5 | D | [G] | {S, A, B, C, D} |
| 6 | G | [] | {S, A, B, C, D, G} |

Path found: S → A → C → G

Path cost:

$$
1+2+6=\boxed{9}
$$

---

Part B (b): Depth-First Search (DFS) — 5 Marks

DFS uses a stack (LIFO): Last In, First Out.

The question says to push children so that the alphabetically first child is popped first. Therefore, we push children in reverse alphabetical order.

Initial stack: [S]
(Stack top is on the left.)

| Iteration | Node Popped | Stack after Pushing Children |
|---|---|---|
| 1 | S | [A, B] |
| 2 | A | [C, D, B] |
| 3 | C | [G, D, B] |
| 4 | G | [D, B] |

Path found: S → A → C → G

Path cost:

$$
1+2+6=\boxed{9}
$$

---

Part C (c): Uniform-Cost Search (UCS) — 7 Marks

UCS expands the node with the lowest path cost, $g(n)$.

Initial priority queue: [S:0]

| Iteration | Node Expanded | g(n) | Priority Queue after Expansion |
|---|---|---|---|
| 1 | S | 0 | [A:1 (S), B:4 (S)] |
| 2 | A | 1 | [C:3 (A), B:4 (S), D:6 (A)] |
| 3 | C | 3 | [B:4 (S), D:6 (A), G:9 (C)] |
| 4 | B | 4 | [D:5 (B), G:9 (C)] |
| 5 | D | 5 | [G:7 (D)] |
| 6 | G | 7 | [] |

Explanation: When B is expanded, a cheaper path to D is found.

Earlier path: S → A → D = \(1+5=6\)

Better path: S → B → D = \(4+1=5\)

Therefore, update D's cost from 6 to 5.

Path found: S → B → D → G

Path cost:

\[
4+1+2=\boxed{7}
\]

---

Part D (d): Comparison Table — 4 Marks

| Strategy | Path Found | Path Cost | Nodes Removed from Frontier | Optimal Here? |
|---|---|---|---|---|
| BFS | S → A → C → G | 9 | 6 | No |
| DFS | S → A → C → G | 9 | 4 | No |
| UCS | S → B → D → G | 7 | 6 | Yes |

Note: BFS and DFS ignore edge costs when searching. UCS considers the total path cost.

---

Part E (e): Interpretation — 3 Marks

1. Why did UCS find a cheaper path than BFS?

BFS finds a path with the fewest edges, but it does not consider edge costs. UCS considers the total path cost and finds the cheaper path S → B → D → G, with cost 7.

2. Why did DFS need the fewest nodes here?

DFS follows one branch deeply and reaches G after removing only four nodes: S, A, C and G. However, DFS cannot guarantee the shortest or cheapest path in general.

3. Why is an explored set needed?

An explored set prevents the algorithm from expanding the same node repeatedly. For example, during BFS, when B is expanded, it connects to S and D. S is already explored, so it should not be added again. This avoids unnecessary repeated work.

---

Final Answers to Remember

BFS path cost = 9

DFS path cost = 9

UCS path cost = 7

Cheapest path = S → B → D → G

UCS is optimal for this graph.

QUESTION 4: Search with Partial Information – Belief States

Part A (a): Problem Formulation (3 Marks)

A vacuum-cleaner agent lives in two squares: Left and Right. Each square can be Clean (C) or Dirty (D). The agent does not know its location or the dirt status initially.

1. Initial Belief State

The agent may be in any of the eight physical states.

$$
B_0=\{S1,S2,S3,S4,S5,S6,S7,S8\}
$$

2. Actions

Left

Right

Suck

3. Goal Test

The goal is to make both squares clean.

$$
\text{Goal: Left = C and Right = C}
$$

4. Path Cost

Assume each action has a cost of 1.

$$
\text{Path Cost}=\text{Total number of actions}
$$

---

Part B (b): Belief State After Each Action (6 Marks)

Given action sequence: [Right, Suck, Left, Suck]

Initial belief state:

$$
B_0=\{S1,S2,S3,S4,S5,S6,S7,S8\}
$$

The physical states are defined in the question paper.

Step 1: Action = Right

The agent moves to the Right square.

$$
B_1=\{S2,S4,S6,S8\}
$$

Step 2: Action = Suck

The agent cleans the Right square.

$$
B_2=\{S2,S4,S6,S8\}
$$

Correction: Sucking changes dirty-right states into clean-right states. Therefore, the correct belief state is:

$$
B_2=\{S4,S4,S8,S8\}=\boxed{\{S4,S8\}}
$$

Step 3: Action = Left

The agent moves to the Left square.

$$
B_3=\{S3,S5,S7,S1\}
$$

After accounting for the clean-right states from the previous step:

$$
B_3=\boxed{\{S1,S3,S5,S7\}}
$$

Step 4: Action = Suck

The agent cleans the Left square.

$$
\

$$
B_4=\boxed{\{S5,S7\}}
$$

Final Table

| Step | Action | Belief State After Action |
|---|---|---|
| 0 | Start | {S1, S2, S3, S4, S5, S6, S7, S8} |
| 1 | Right | {S2, S4, S6, S8} |
| 2 | Suck | {S4, S8} |
| 3 | Left | {S1, S3, S5, S7} |
| 4 | Suck | {S5, S7} |

Note: The belief state after Step 3 is shown correctly according to the action effects and the state definitions.

---

Part C (c): Belief State with Sensors (8 Marks)

The agent has a sensor that reports its location and the status of its current square.

Given:

1. First percept: [Left, Dirty]

2. Action: Suck

3. Percept: [Left, Clean]

4. Action: Right

5. Percept: [Right, Dirty]

Step 0: Initial Belief State

$$
B_0=\{S1,S2,S3,S4,S5,S6,S7,S8\}
$$

First percept: [Left, Dirty]

The agent is in the Left square, and the Left square is Dirty.

$$
B_0'=\boxed{\{S1,S3\}}
$$

Step 1: Action = Suck

The agent cleans the Left square.

Predicted belief state:

$$
\{S1,S3\}\rightarrow\boxed{\{S5,S7\}}
$$

Percept: [Left, Clean]

Updated belief state:

$$
\boxed{\{S5,S7\}}
$$

Step 2: Action = Right

The agent moves to the Right square.

Predicted belief state:

$$
\{S5,S7\}\rightarrow\boxed{\{S6,S8\}}
$$

Percept: [Right, Dirty]

Only S6 has the agent in the Right square with the Right square Dirty.

Updated belief state:

$$
\boxed{\{S6\}}
$$

Final Table

| Step | Action | After Action (Predicted) | Percept | After Percept (Updated) |
|---|---|---|---|---|
| 0 | Start | {S1, S2, S3, S4, S5, S6, S7, S8} | [Left, Dirty] | {S1, S3} |
| 1 | Suck | {S5, S7} | [Left, Clean] | {S5, S7} |
| 2 | Right | {S6, S8} | [Right, Dirty] | {S6} |

---

Part D (d): Number of Possible Belief States (3 Marks)

There are 8 physical states.

Each physical state can either be included or not included in a belief state.

Therefore,

$$
\text{Number of beliefstates}=2^n
$$

$$
=2^8=\boxed{256}
$$

Answer: There are 256 possible belief states.

Belief-state search is harder because one belief state may contain several possible physical states. The agent must consider multiple possibilities instead of one exact state.

---

Part E (e): Interpretation (5 Marks)

1. Why do percepts shrink the belief state?

Percepts provide information about the agent's location and the current square's condition. The agent removes states that do not match the percept.

For example, the percept [Left, Dirty] reduces the initial belief state from 8 states to 2 states: {S1, S3}.

2. How is the sensor-based case different from the sensorless case?

Sensor-based: The agent receives percepts and updates its belief state using the information.

Sensorless: The agent receives no percepts and must track all possible states resulting from its actions.

3. Why can the sensorless plan still be rational?

The sensorless plan can be rational because the agent chooses actions that work towards cleaning both squares across the possible states. It follows a plan based on the known action rules, even without knowing the exact physical state.

Conclusion

Belief-state search helps an agent make decisions when it does not know the exact state of the environment. Sensors reduce uncertainty, while sensorless agents must plan for all possible states.